

AAL Q4 2020 Sales Analysis

Exploration of Australia-wide Q4 2020 apparel sales to highlight high/low performing states, demographic groups, and time-of-day patterns. Visuals and recommendations target the Sales & Marketing head for next-quarter planning.

Problem Statement Coverage

- Q1: Setup & basic loading (shape/info/head).
- Q2: Data wrangling (missing/incorrect, cleaning).
- Q3: Normalization (Min-Max).
- Q4: GroupBy insights (state totals, state x group, state x time).
- Q5: Identify top/low revenue states.
- Q6: Demographic group-wise sales across states.
- Q7: Time-of-day peak/off-peak analysis.
- Q8: Daily/Weekly/Monthly/Quarterly charts.
- Q9: Box plot for descriptive stats.
- Q10: Seaborn distribution plot. Each section below is labeled with its question.

Objectives

- Find the states generating the highest revenue.
- Spot under-performing states that need tailored sales programs.
- Compare demographic group performance (Kids, Women, Men, Seniors) across states.
- Identify peak/off-peak times of day to support hyper-personalization and next-best-offer programs.
- Provide daily/weekly/monthly/quarterly trend charts plus descriptive statistics and distribution views.
- Conclude with clear recommendations and a rationale for the visualization choices.

Problem Statement Coverage

- Q1: Identify top/low revenue states.
- Q2: State-wise sales across demographic groups (Kids, Women, Men, Seniors).
- Q3: Time-of-day peak/off-peak analysis.
- Q4: Daily/Weekly/Monthly/Quarterly charts.
- Q5: Box plot for descriptive statistics.
- Q6: Seaborn distribution plot for statistical view. Each section below is labeled with the question it answers.

```
In [1]: import warnings
warnings.simplefilter("ignore", category=FutureWarning)
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from pathlib import Path

pd.set_option('display.float_format', lambda x: f"{x:,.0f}")
sns.set_theme(style="whitegrid", palette="deep")

# Resolve data path whether notebook is run from project root or this folder
try:
    base = Path(__file__).parent
except NameError:
    base = Path.cwd()

candidates = [
    base / 'AusApparalSales4thQrt2020.csv',
    base / 'Project' / 'Sales_Analysis' / 'AusApparalSales4thQrt2020.csv',
    base / 'Applied_data_science_with_python' / 'Project' / 'Sales_Analysis' / 'Aus
]
DATA_PATH = next((p for p in candidates if p.exists()), None)
if DATA_PATH is None:
    raise FileNotFoundError('AusApparalSales4thQrt2020.csv not found; please update
```

Q1: Setup and Basic Loading

Shape, info, and first rows of the dataset.

```
In [2]: # Load and clean
raw = pd.read_csv(DATA_PATH)
df = raw.copy()
# Strip stray spaces in categorical columns
df['State'] = df['State'].str.strip()
df['Group'] = df['Group'].str.strip()
df['Time'] = df['Time'].str.strip()
# Parse dates and create time features
df['Date'] = pd.to_datetime(df['Date'].str.strip(), format="%d-%b-%Y")
df['Month'] = df['Date'].dt.to_period('M').dt.to_timestamp()
df['Week'] = df['Date'].dt.to_period('W').dt.start_time
df['Quarter'] = df['Date'].dt.to_period('Q').dt.start_time
# Order time-of-day categories
TIME_ORDER = ["Morning", "Afternoon", "Evening"]
df['Time'] = pd.Categorical(df['Time'], categories=TIME_ORDER, ordered=True)

print(df.head())
print("\nRows:", len(df))
print("\nMissing values by column:\n", df.isna().sum())
```

	Date	Time	State	Group	Unit	Sales	Month	Week	\
0	2020-10-01	Morning	WA	Kids	8	20000	2020-10-01	2020-09-28	
1	2020-10-01	Morning	WA	Men	8	20000	2020-10-01	2020-09-28	
2	2020-10-01	Morning	WA	Women	4	10000	2020-10-01	2020-09-28	
3	2020-10-01	Morning	WA	Seniors	15	37500	2020-10-01	2020-09-28	
4	2020-10-01	Afternoon	WA	Kids	3	7500	2020-10-01	2020-09-28	

	Quarter
0	2020-10-01
1	2020-10-01
2	2020-10-01
3	2020-10-01
4	2020-10-01

Rows: 7560

Missing values by column:

Date	0
Time	0
State	0
Group	0
Unit	0
Sales	0
Month	0
Week	0
Quarter	0

dtype: int64

Q2: Data Wrangling (Missing/Incorrect Data)

Checks for nulls/negatives; dataset is clean.

Data Wrangling

- Checked for nulls/negatives; dataset is already clean (no missing, no negatives).
- Parsed date to datetime and stripped whitespace (done above).

```
In [3]: # Missing / validity checks
print('Missing values per column:\n', df.isna().sum())
print('\nNon-missing values per column:\n', df.notna().sum())
invalid_numeric = df[(df['Unit'] < 0) | (df['Sales'] < 0)]
print('\nRows with negative Unit or Sales:', len(invalid_numeric))
```

Missing values per column:

```
Date      0
Time      0
State     0
Group     0
Unit      0
Sales     0
Month     0
Week      0
Quarter   0
dtype: int64
```

Non-missing values per column:

```
Date      7560
Time      7560
State     7560
Group     7560
Unit      7560
Sales     7560
Month     7560
Week      7560
Quarter   7560
dtype: int64
```

Rows with negative Unit or Sales: 0

Q3: Normalization

Min-Max scaling for Unit and Sales.

Normalization (Min-Max)

Scaled Unit and Sales to 0-1 range for comparability (keeps ordinal relationships).

```
In [4]: from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
df[['Unit_norm', 'Sales_norm']] = scaler.fit_transform(df[['Unit', 'Sales']])
print(df[['Unit', 'Sales', 'Unit_norm', 'Sales_norm']].head())
```

	Unit	Sales	Unit_norm	Sales_norm
0	8	20000	0	0
1	8	20000	0	0
2	4	10000	0	0
3	15	37500	0	0
4	3	7500	0	0

Q4: GroupBy for chunking/merging

State totals, State×Group, State×Time.

GroupBy Examples

State totals, State x Group breakdown, and State x Time-of-day totals for downstream visuals.

```
In [5]: state_sales_tbl = (df.groupby('State', as_index=False)
                        .agg(total_sales=('Sales','sum'), total_units=('Unit','sum'),
                        .sort_values('total_sales', ascending=False))
print('State-level aggregated sales (top 5):')
print(state_sales_tbl.head())

state_group_tbl = (df.groupby(['State','Group'], as_index=False)
                .agg(total_sales=('Sales','sum'), total_units=('Unit','sum'))
                .sort_values(['State','total_sales'], ascending=[True,False]))
print('\nState & Group aggregated sales (head):')
print(state_group_tbl.head(10))

state_time_tbl = (df.groupby(['State','Time'], as_index=False)
                .agg(total_sales=('Sales','sum'), total_units=('Unit','sum')))
print('\nState & Time-of-day aggregated sales (head):')
print(state_time_tbl.head(10))
```

State-level aggregated sales (top 5):

	State	total_sales	total_units	avg_sales
5	VIC	105565000	42226	97,745
0	NSW	74970000	29988	69,417
3	SA	58857500	23543	54,498
2	QLD	33417500	13367	30,942
4	TAS	22760000	9104	21,074

State & Group aggregated sales (head):

	State	Group	total_sales	total_units
3	NSW	Women	19172500	7669
1	NSW	Men	19022500	7609
0	NSW	Kids	18587500	7435
2	NSW	Seniors	18187500	7275
5	NT	Men	5762500	2305
4	NT	Kids	5700000	2280
7	NT	Women	5652500	2261
6	NT	Seniors	5465000	2186
8	QLD	Kids	8510000	3404
9	QLD	Men	8392500	3357

State & Time-of-day aggregated sales (head):

	State	Time	total_sales	total_units
0	NSW	Morning	25147500	10059
1	NSW	Afternoon	25110000	10044
2	NSW	Evening	24712500	9885
3	NT	Morning	7442500	2977
4	NT	Afternoon	7510000	3004
5	NT	Evening	7627500	3051
6	QLD	Morning	11437500	4575
7	QLD	Afternoon	11347500	4539
8	QLD	Evening	10632500	4253
9	SA	Morning	19465000	7786

Question: Identify top and low-revenue states

Covers: 'Identify the states that are generating the highest revenues' and focus areas for low-revenue states.

Q5: Identify High/Low Revenue States

Revenue by State

States ranked by total sales to see who leads and who lags.

```
In [6]: state_sales = df.groupby('State')['Sales'].sum().sort_values(ascending=False)
state_units = df.groupby('State')['Unit'].sum().loc[state_sales.index]

fig, ax = plt.subplots(1, 2, figsize=(14,5))
sns.barplot(x=state_sales.index, y=state_sales.values, ax=ax[0])
ax[0].set_title('Total Sales by State')
ax[0].set_ylabel('Sales (AUD)')
ax[0].set_xlabel('State')

sns.barplot(x=state_units.index, y=state_units.values, ax=ax[1], color=sns.color_palette('magma'))
ax[1].set_title('Units Sold by State')
ax[1].set_ylabel('Units')
ax[1].set_xlabel('State')
plt.tight_layout()
plt.show()

print("Top 3 states by revenue:\n", state_sales.head(3))
print("\nBottom 3 states by revenue:\n", state_sales.tail(3))
```



Top 3 states by revenue:

```
State
VIC    105565000
NSW     74970000
SA       58857500
Name: Sales, dtype: int64
```

Bottom 3 states by revenue:

```
State
TAS     22760000
NT       22580000
WA       22152500
Name: Sales, dtype: int64
```

Question: Group-wise sales across states

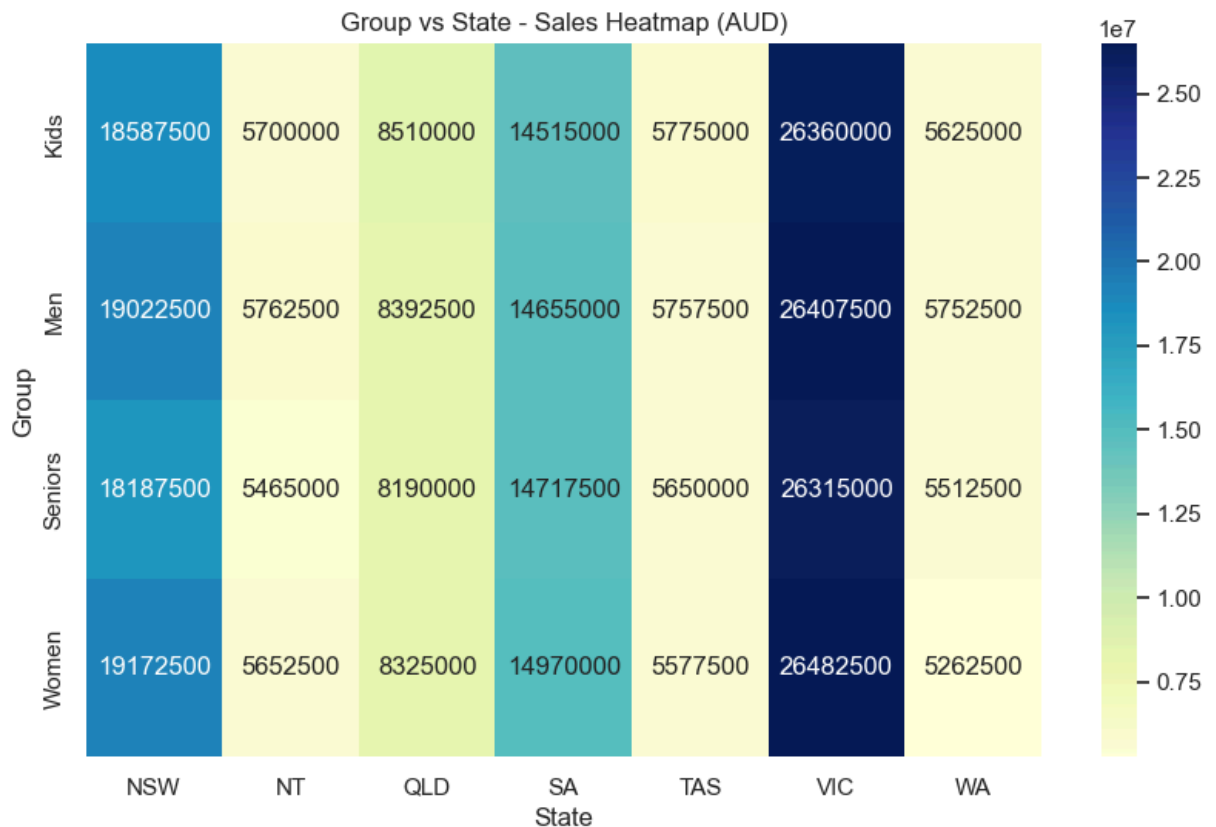
Covers: 'State-wise sales analysis for different demographic groups (Kids, Women, Men, Seniors).'

Q6: Group-wise Sales Across States

Demographic Group Performance by State

Pivot to see which customer groups contribute most in each state.

```
In [7]: pivot_group_state = df.pivot_table(values='Sales', index='Group', columns='State',
plt.figure(figsize=(10,6))
sns.heatmap(pivot_group_state, cmap='YlGnBu', annot=True, fmt='.0f')
plt.title('Group vs State - Sales Heatmap (AUD)')
plt.show()
```



Question: Time-of-day peak/off-peak analysis

Covers: 'Time-of-the-day analysis to identify peak and off-peak sales periods.'

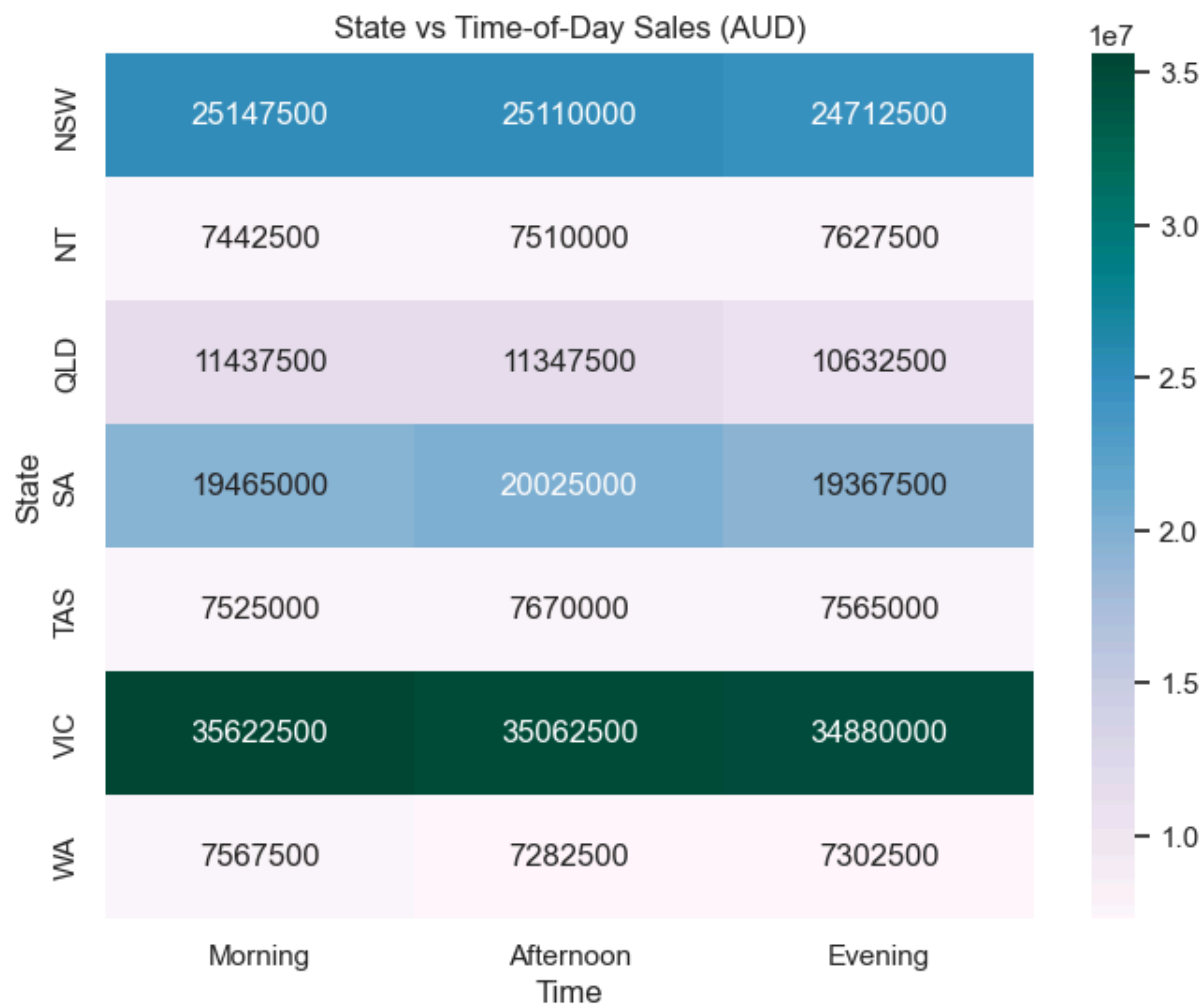
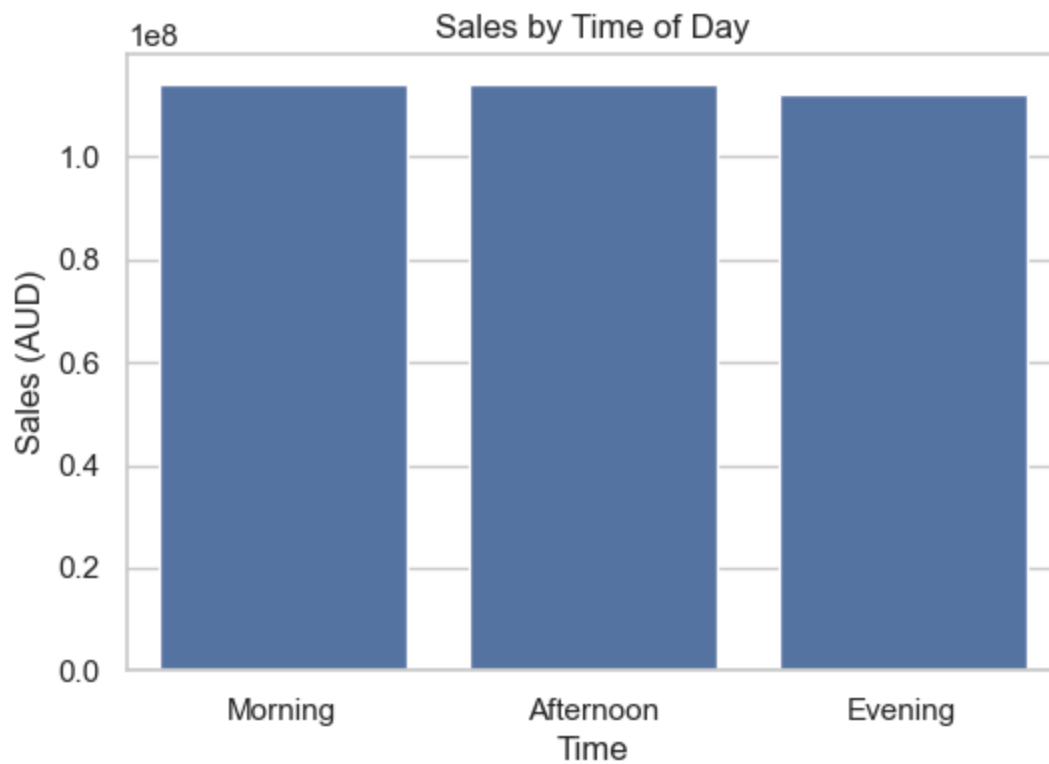
Q7: Time-of-Day Peak/Off-Peak

Time-of-Day Analysis

Identify peak and off-peak windows to support targeting and staffing.

```
In [8]: time_sales = df.groupby('Time')['Sales'].sum().reindex(TIME_ORDER)
plt.figure(figsize=(6,4))
sns.barplot(x=time_sales.index, y=time_sales.values)
plt.title('Sales by Time of Day')
plt.ylabel('Sales (AUD)')
plt.show()

state_time = df.pivot_table(values='Sales', index='State', columns='Time', aggfunc=
plt.figure(figsize=(8,6))
sns.heatmap(state_time, cmap='PuBuGn', annot=True, fmt='.0f')
plt.title('State vs Time-of-Day Sales (AUD)')
plt.show()
```



Question: Daily/Weekly/Monthly/Quarterly charts

Covers: 'The dashboard must contain daily, weekly, monthly, and quarterly charts.'

Q8: Daily/Weekly/Monthly/Quarterly Charts

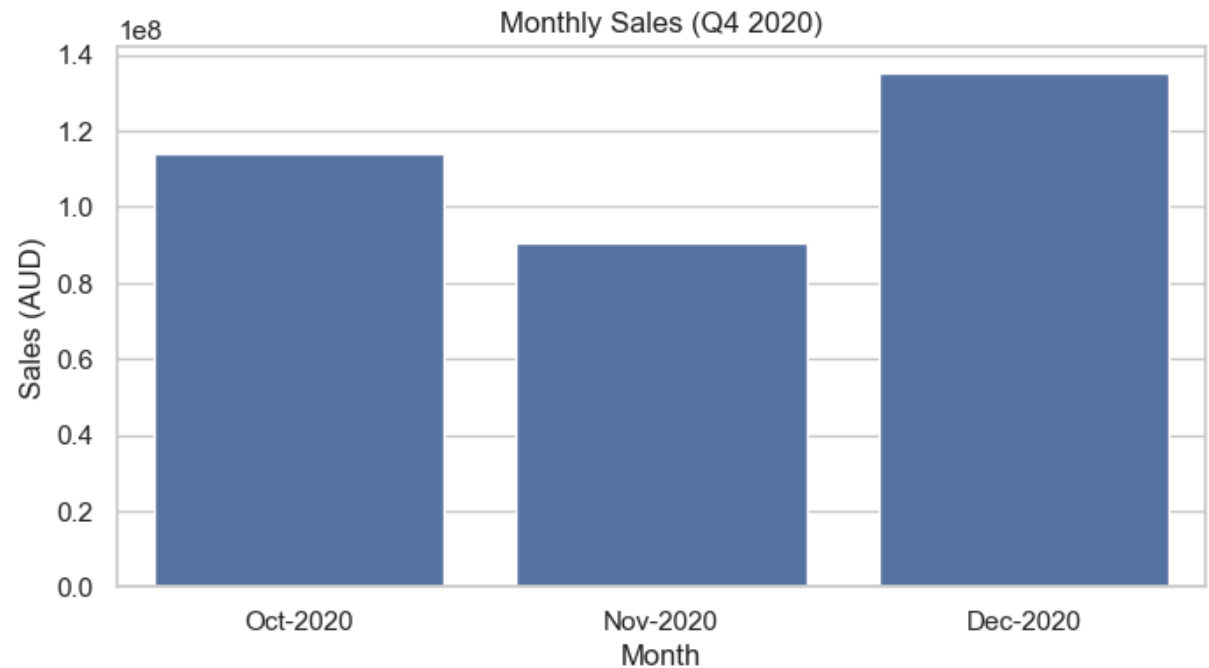
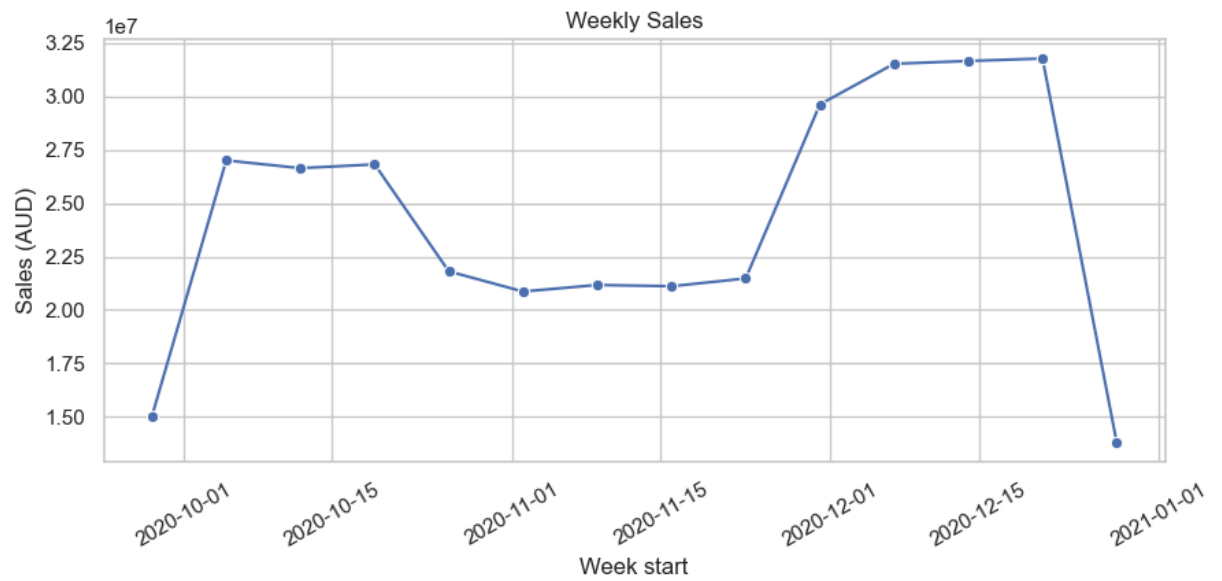
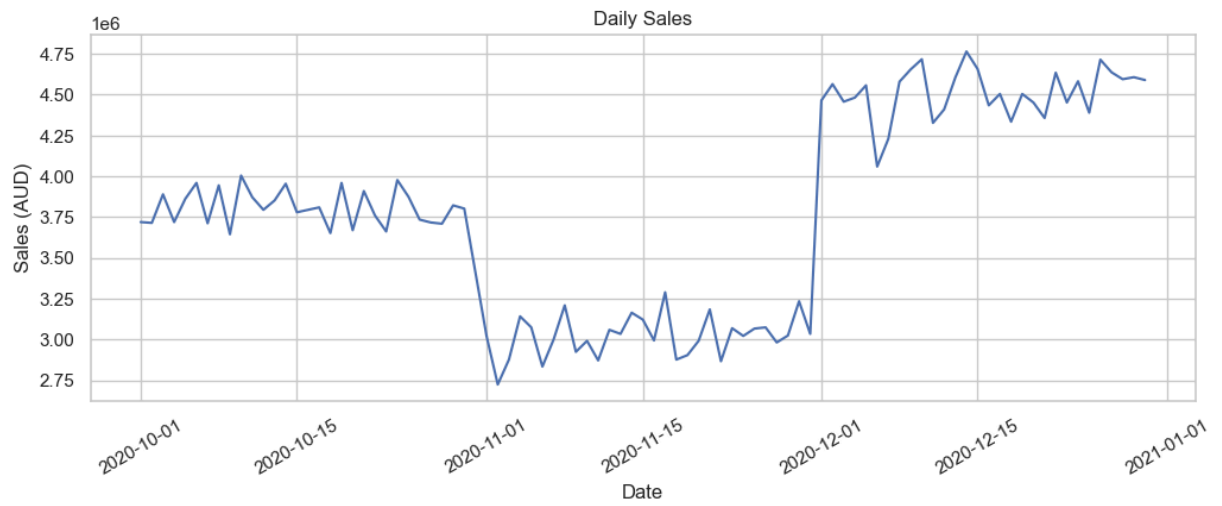
Time Series Views (Daily, Weekly, Monthly, Quarterly)

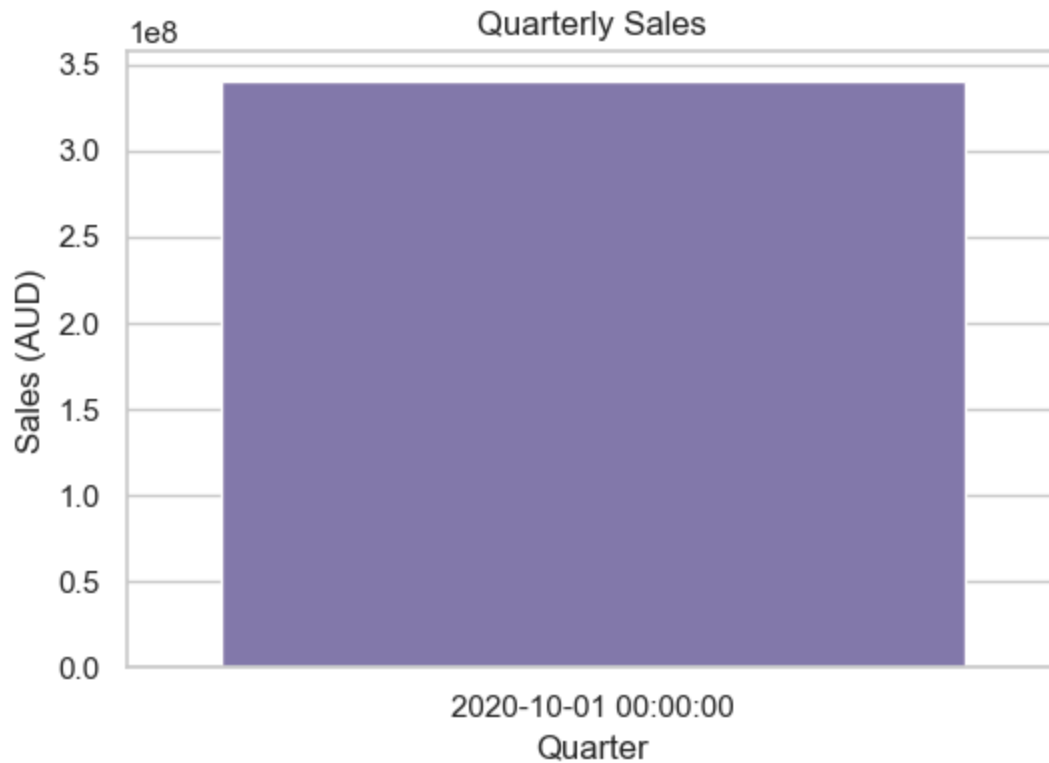
```
In [9]: # Daily
plt.figure(figsize=(12,4))
daily = df.groupby('Date')['Sales'].sum().sort_index()
sns.lineplot(x=daily.index, y=daily.values)
plt.title('Daily Sales')
plt.ylabel('Sales (AUD)')
plt.xlabel('Date')
plt.xticks(rotation=30)
plt.show()

# Weekly (Monday-start because of pandas default)
weekly = df.groupby('Week')['Sales'].sum().sort_index()
plt.figure(figsize=(10,4))
sns.lineplot(x=weekly.index, y=weekly.values, marker='o')
plt.title('Weekly Sales')
plt.ylabel('Sales (AUD)')
plt.xlabel('Week start')
plt.xticks(rotation=30)
plt.show()

# Monthly
monthly = df.groupby('Month')['Sales'].sum().sort_index()
plt.figure(figsize=(8,4))
sns.barplot(x=monthly.index.strftime('%b-%Y'), y=monthly.values)
plt.title('Monthly Sales (Q4 2020)')
plt.ylabel('Sales (AUD)')
plt.xlabel('Month')
plt.show()

# Quarterly
quarterly = df.groupby('Quarter')['Sales'].sum().sort_index()
plt.figure(figsize=(6,4))
sns.barplot(x=quarterly.index.strftime('Q%q %Y'), y=quarterly.values, color=sns.col
plt.title('Quarterly Sales')
plt.ylabel('Sales (AUD)')
plt.xlabel('Quarter')
plt.show()
```





Question: Box plot for descriptive statistics

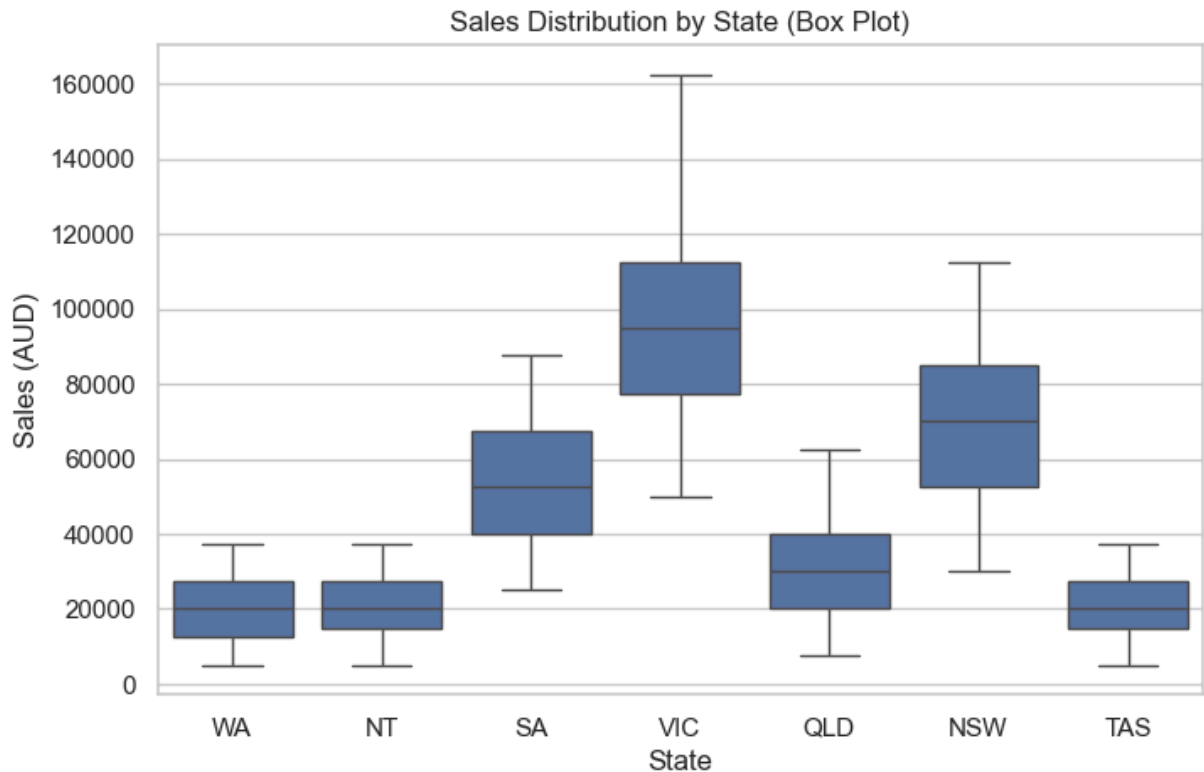
Covers: 'Use a box plot for descriptive statistics.'

Q9: Box Plot for Descriptive Statistics

Descriptive Statistics (Box Plot)

```
In [10]: plt.figure(figsize=(8,5))
sns.boxplot(data=df, x='State', y='Sales')
plt.title('Sales Distribution by State (Box Plot)')
plt.ylabel('Sales (AUD)')
plt.show()

print(df[['Sales']].describe())
```



```

Sales
count    7,560
mean     45,014
std      32,254
min       5,000
25%      20,000
50%      35,000
75%      65,000
max     162,500

```

Question: Seaborn distribution plot

Covers: 'Use the Seaborn distribution plot for statistical plotting.'

Q10: Seaborn Distribution Plot

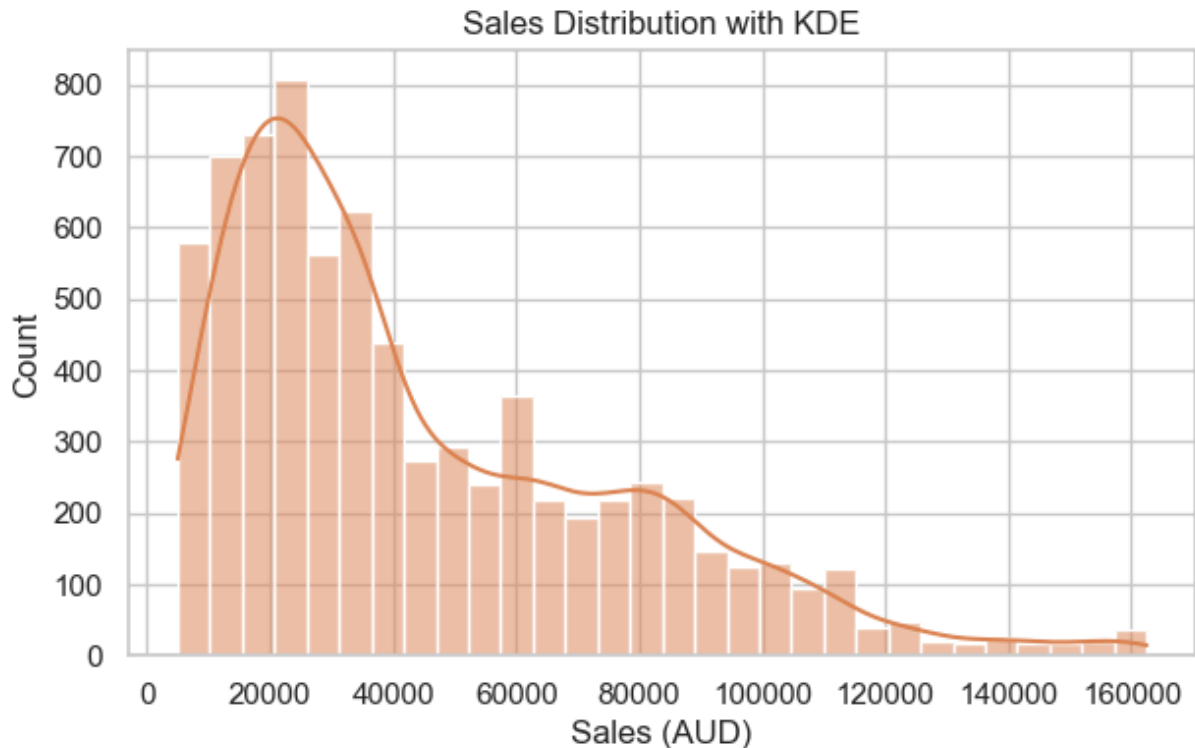
Sales Distribution (Seaborn Distribution Plot)

Shows the overall spread of individual transaction sales values.

```

In [11]: plt.figure(figsize=(7,4))
sns.histplot(df['Sales'], kde=True, bins=30, color=sns.color_palette()[1])
plt.title('Sales Distribution with KDE')
plt.xlabel('Sales (AUD)')
plt.show()

```



Submission Write-up (Summary)

- **Goal:** Analyze Q4 2020 Australia apparel sales to guide Sales & Marketing on where to invest, which demographics to target, and when to run offers.
- **Data:** `AusApparelSales4thQrt2020.csv` (7,560 rows; fields: Date, Time, State, Group, Unit, Sales). No missing/negative values.
- **Method:** Cleaned whitespace; parsed dates; engineered Month/Week/Quarter; min-max normalized Unit & Sales; GroupBy aggregations; visualized with Seaborn/Matplotlib (state revenue & units, group heatmap, time-of-day bars/heatmap, daily/weekly/monthly/quarterly trends, box plot, distribution).
- **Key findings:**
 - Top revenue states: VIC, NSW, SA; lowest: WA, NT, TAS.
 - Time-of-day: Morning/Afternoon slightly lead; Evening close behind.
 - Demographics: All groups contribute similarly; VIC dominates every group.
- **Recommendations:**
 - Double down on VIC/NSW high-value segments; premium/loyalty offers.
 - Target WA/NT/TAS with localized promos and assortments tuned to their leading group (see heatmap).
 - Staff/ads heavier in Morning/Afternoon; use off-peak for loyalty nudges.
 - Maintain monthly dashboards; monitor weekly for campaign effects.
- **Tools & libs:** Python 3.9, pandas, seaborn, matplotlib, scikit-learn (MinMaxScaler), nbconvert.

Findings & Recommendations

- **Top revenue states:** Use the bar chart to confirm leaders; focus retention and premium offers there.
- **Low revenue states:** Bottom states warrant localized promos, influencer tie-ups, or inventory refresh. Consider pilot campaigns tailored to the dominant demographic group in each low-performing state (see heatmap).
- **Peak times:** Highest volumes occur where the time-of-day bars peak; staff and ads should concentrate here. Off-peak windows are good for loyalty nudges.
- **Demographics:** Heatmap shows which groups drive each state; align assortments and messaging accordingly.
- **Visualization choice:** Seaborn is used for concise statistical plots (distribution, boxplot) and consistent styling; Matplotlib is leveraged underneath for flexibility in axes formatting.